УЧЕБНАЯ ПРАКТИКА

Инвентарь

Система учёта оборудования и аудиторий колледжа

Общая проектная документация и отчёт о проверках

Участники: Самир и Артём
Версия документа: 21 октября 2026 года

Для чего нужен проект

Приложение помогает найти технику, подать заявку, записать выдачу и возврат, увидеть перемещения и проверить наличие оборудования в аудитории. Все рабочие записи сохраняются в базе данных.

Роль	Что может делать
Преподаватель	Искать технику, подавать заявки, смотреть свои выдачи и профиль.
Лаборант	Вести оборудование и аудитории, рассматривать заявки, выдавать и принимать технику, проводить инвентаризацию, скачивать XLSX.
Администратор	Работать с техникой, управлять ролями и блокировкой пользователей, читать общий журнал действий.

Что использовано

React и TypeScript - страницы сайта. Express на Node.js - обработка запросов. PostgreSQL и Prisma - хранение данных. Docker Compose - совместный запуск. GitHub Actions - автоматические проверки.

Ссылки на проект

Репозиторий GitHub · Рабочий сайт · Swagger: описание API

Это один общий документ команды. Индивидуальные отчёты и дневники каждого студента готовятся отдельно.

Пользовательские истории: 1-7

Каждая история отвечает на три вопроса: кто пользуется функцией, что он хочет сделать и зачем. Ниже - простые условия проверки.

US-01. Как преподаватель, я хочу создать аккаунт и войти, чтобы пользоваться приложением.

Проверка: Нельзя зарегистрировать один email дважды. Новый пользователь получает роль преподавателя. Пароль не хранится открытым текстом. Вход, продление сеанса и выход работают.

US-02. Как пользователь, я хочу восстановить пароль, чтобы снова войти в аккаунт.

Проверка: Ссылка восстановления действует ограниченное время и только один раз. Для учебного проекта она выводится в журнал сервера.

US-03. Как преподаватель, я хочу найти оборудование, чтобы выбрать технику для занятия.

Проверка: Сервер ищет записи, применяет фильтры, меняет порядок и делит список на страницы. Если ничего не найдено или произошла ошибка, это видно на экране.

US-04. Как лаборант, я хочу добавлять и менять оборудование, чтобы сведения о технике были верными.

Проверка: Можно открыть, добавить, изменить и удалить запись по своим правам. Инвентарные номера не повторяются. Перед удалением нужно подтверждение. Изменения записываются в историю.

US-05. Как лаборант, я хочу прикрепить фото или документ, чтобы дополнить описание техники.

Проверка: Можно загрузить PNG, JPG или PDF до 5 МБ. Приложение и сервер проверяют файл. Скачать и удалить его можно только по своим правам.

US-06. Как преподаватель, я хочу отправить заявку, чтобы получить технику для занятия.

Проверка: В заявке есть от 1 до 20 разных предметов. Пустая заявка и повтор одного предмета запрещены. Заявка сохраняется за её автором.

US-07. Как лаборант, я хочу одобрить или отклонить заявку, чтобы решить, выдавать ли технику.

Проверка: Решение видно преподавателю. Повторное решение запрещено. Преподаватель не может одобрять заявки. Само одобрение ещё не делает технику занятой.

Пользовательские истории: 8-15

US-08. Как лаборант, я хочу записать выдачу и возврат, чтобы знать, у кого находится техника.

Проверка: При выдаче задаётся срок возврата. Занятую технику нельзя выдать снова. Весь комплект выдаётся или возвращается целиком: ошибка по одному предмету отменяет действие. Повторный возврат запрещён.

US-09. Как лаборант, я хочу менять аудиторию оборудования, чтобы знать, где оно находится.

Проверка: Аудитории можно добавлять и менять по своим правам. В истории перемещения видны старая и новая аудитории, автор и дата.

US-10. Как лаборант, я хочу проверить оборудование в аудитории, чтобы найти недостающие предметы.

Проверка: Есть список предметов и отметки о наличии. Результат, автор и дата сохраняются. Готовую проверку можно открыть в истории, но нельзя изменить.

US-11. Как пользователь, я хочу изменить профиль и посмотреть свои действия, чтобы управлять аккаунтом.

Проверка: Можно изменить имя, email и пароль. Неверные данные отклоняются. История относится только к своему аккаунту. Подстановка чужого номера записи не открывает чужие данные.

US-12. Как администратор, я хочу менять роли и блокировать пользователей, чтобы управлять доступом.

Проверка: Роль можно изменить, пользователя можно заблокировать и разблокировать. После смены доступа старые сеансы закрываются. Последнего активного администратора отключить нельзя.

US-13. Как лаборант, я хочу видеть количество и состояние техники, чтобы понимать, что есть в наличии.

Проверка: В обзоре есть счётчики и график. Числа берутся из базы данных. Преподаватель видит только свои заявки и выдачи, сотрудники - общую сводку.

US-14. Как администратор, я хочу смотреть журнал действий и скачивать таблицу оборудования, чтобы проверять работу и готовить отчёты.

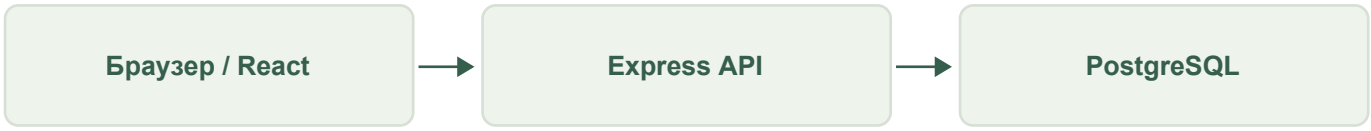
Проверка: В журнале видны автор, действие и дата. В XLSX попадают все предметы с выбранными фильтрами, а не одна страница. Файл открывается как таблица. Доступ проверяет сервер.

US-15. Как пользователь, я хочу выбрать тему и язык, чтобы мне было удобно пользоваться сайтом.

Проверка: Есть светлая, тёмная и системная темы, русский и английский языки. Выбор сохраняется. Страницы работают на телефоне и компьютере, от 360 до 1920 пикселей.

Как работает приложение

Пользователь нажимает кнопку в браузере. Страница отправляет запрос серверу. Сервер проверяет права, читает или изменяет данные и возвращает результат.



На публичном сервере Caddy обеспечивает HTTPS, а Nginx передаёт запросы к API. Внутри API контроллер принимает запрос, сервис проверяет правила, репозиторий обращается к базе.

Вход и безопасность

Пароли хранятся в виде bcrypt-хешей. После входа используются access JWT и refresh JWT: первый подтверждает текущий сеанс, второй помогает продлить его. Refresh хранится в защищённой HttpOnly cookie. Выход и смена пароля закрывают старые сеансы.

Права проверяет сервер. Преподаватель не получает чужие заявки даже при подстановке номера записи. Запросы к БД выполняются через Prisma, пользовательский текст выводится как текст. Для файлов проверяются тип и размер.

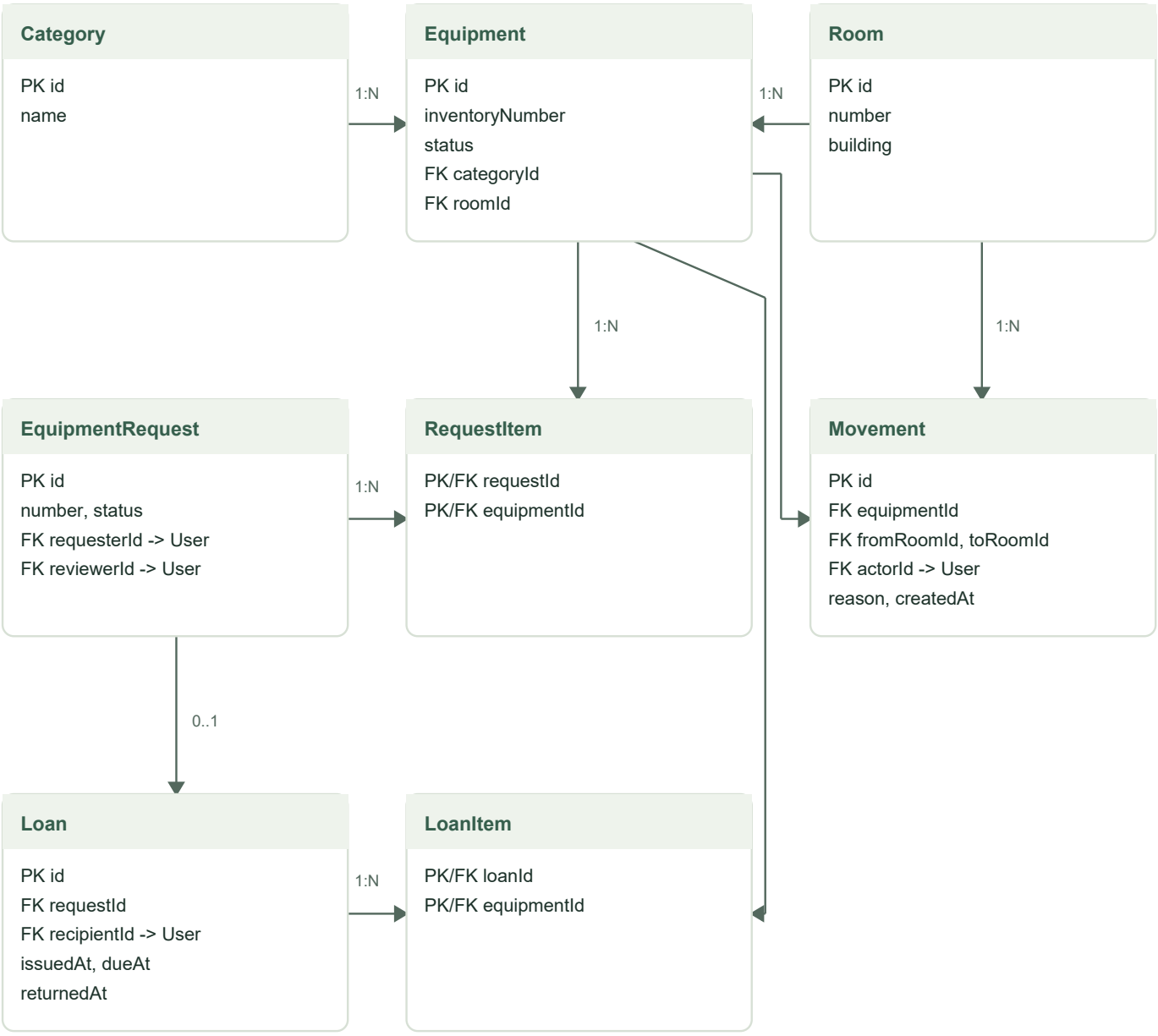
Основные запросы API

Пример запроса	Для чего нужен
POST /api/auth/login	Войти в аккаунт.
GET /api/equipment	Получить каталог с поиском и фильтрами.
POST /api/requests	Отправить заявку.
POST /api/loans	Оформить выдачу.
GET /api/dashboard	Получить данные обзора.
GET /api/equipment/export	Скачать XLSX.
GET /api/audit	Прочитать журнал администратора.

API описано в OpenAPI 3. Полный список из 38 путей находится в Swagger и во вложении openapi.json этого PDF. Ошибки возвращаются в одном формате: код и понятное сообщение.

Схема БД: оборудование и выдачи

В базе 16 таблиц. Здесь показаны восемь таблиц предметной области и их основные поля. Полная схема Prisma приложена к PDF отдельным файлом.

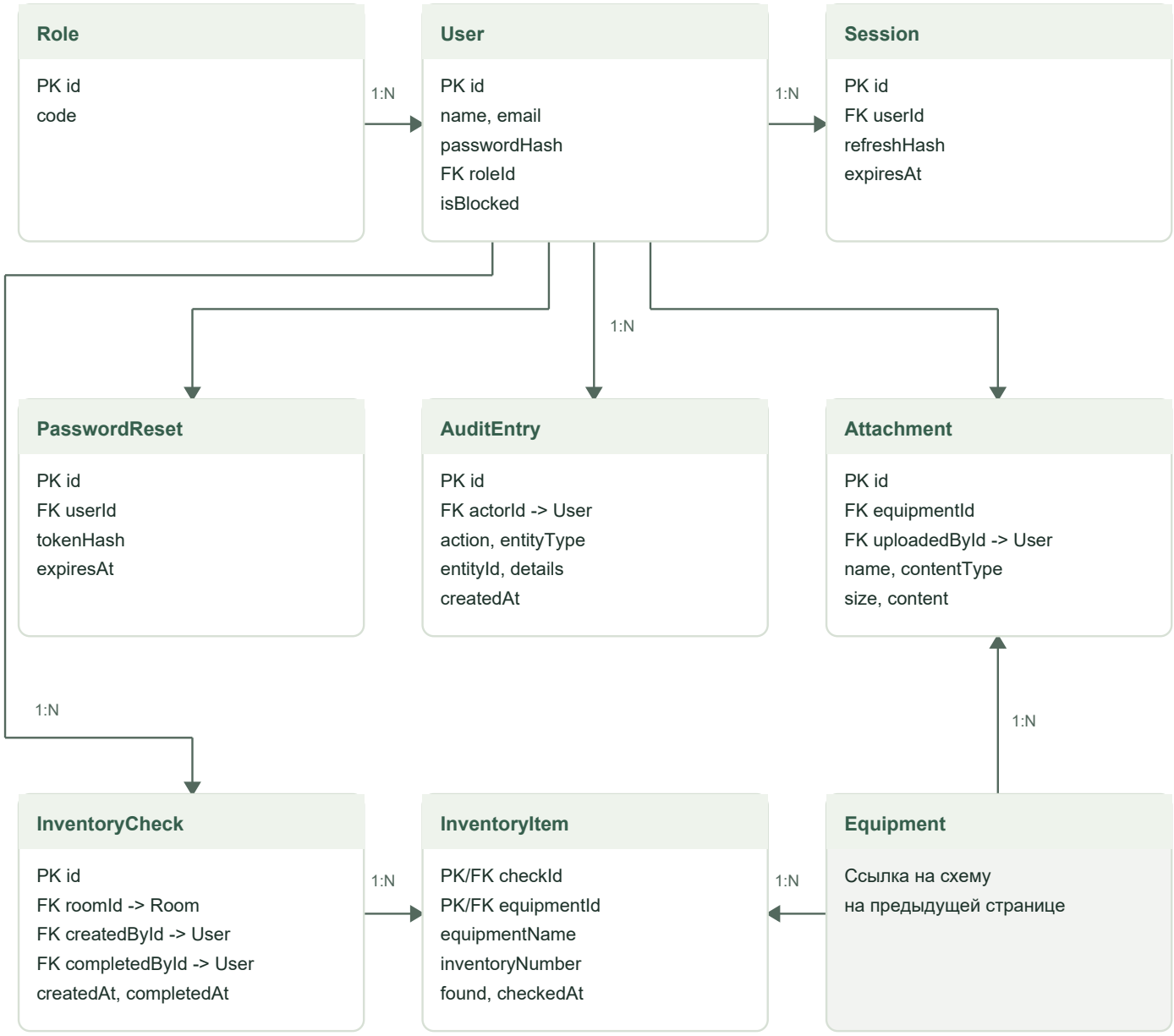


Как читать: PK - уникальный номер записи. FK - ссылка на другую таблицу. 1:N означает «одна запись связана со многими». 0..1 - необязательная связь с одной записью.

Категория и аудитория содержат много предметов. Заявка и выдача могут включать несколько предметов: RequestItem и LoanItem связывают их с оборудованием. Такие промежуточные таблицы реализуют связь «многие ко многим». Поля requesterId, recipientId и actorId ссылаются на User на следующей странице.

Схема БД: аккаунты и проверки

Остальные восемь таблиц отвечают за пользователей, сеансы, историю, файлы и инвентаризацию. Серый блок Equipment - ссылка на таблицу с предыдущей страницы.

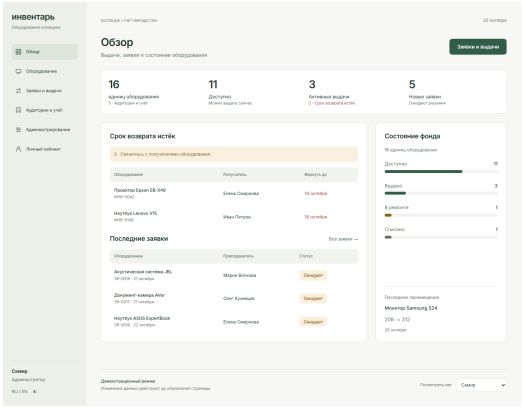


Role задаёт роль пользователя. Session и PasswordReset нужны для входа и восстановления пароля. AuditEntry хранит действия. Attachment хранит вложения. InventoryCheck - сама проверка аудитории, InventoryItem - отметки по её предметам.

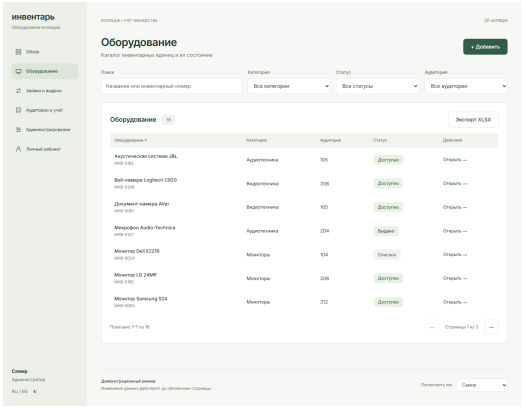
У InventoryCheck также есть ссылки на аудиторию и авторов. У выдачи есть ссылки на выдавшего и принявшего сотрудника. Все эти связи и индексы полностью перечислены в приложенном schema.prisma. Изменения схемы применяются миграциями.

Основные экраны

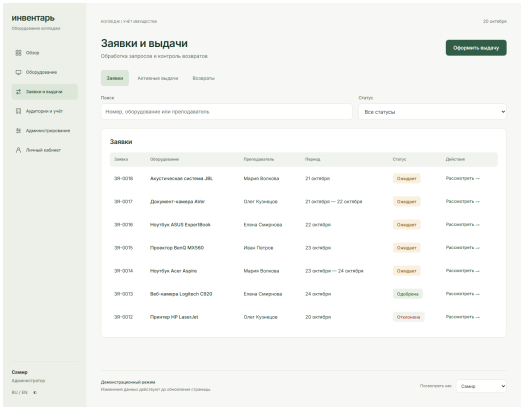
Ниже - примеры интерфейса с демонстрационными данными. В рабочем приложении записи сохраняются в PostgreSQL. В Figma есть шесть основных экранов, светлая и тёмная темы, мобильные варианты.



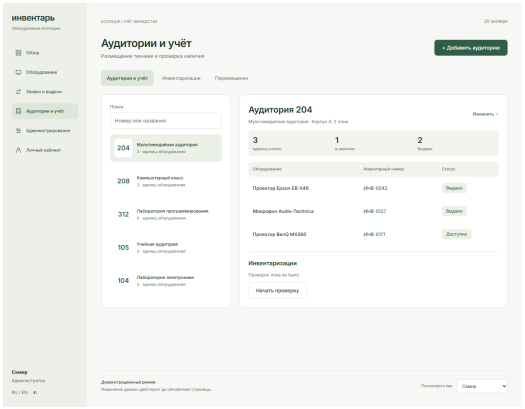
Обзор



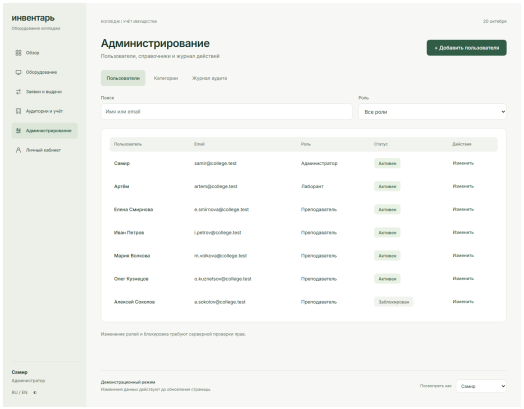
Каталог оборудования



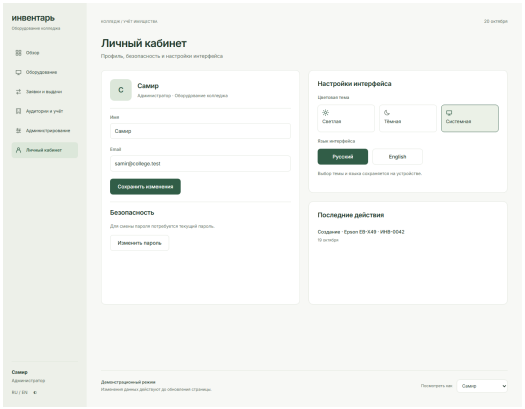
Заявки и выдачи



Аудитории



Администрирование



Профиль

Открыть макеты в Figma. Язык и тема сохраняются в браузере. Проверены ширины 360, 768, 1440 и 1920 пикселей.

Проверки и запуск

Проверка	Результат
Серверные тесты	92 прошли. Проверяются правила работы и API с отдельной PostgreSQL.
Тесты интерфейса	50 прошли. Проверяются формы, списки, ошибки и права.
Браузерные сценарии	21 прошёл с настоящим API.
Покрытие сервисов	97,3% строк при минимуме задания 60%.
Код и запуск	Линтер, форматирование, TypeScript, сборки и Docker Compose прошли в CI.

Что проверено на практике

Вход и восстановление, роли, каталог, заявки, выдача и возврат комплекта, файлы, инвентаризация, блокировка, история, обзор и XLSX. На публичном HTTPS-сайте проверено скачивание таблиц RU/EN и запрет экспорта преподавателю. Типовые запросы локального API на seed-данных укладывались в 23 мс после прогрева; требование - до 500 мс.

Какие ошибки исправили

История была привязана к имени - теперь используется ID. Частичное изменение могло сбрасывать пропущенные поля - схема исправлена. Удаление связанной записи могло давать 500 - теперь возвращается понятный отказ 409. За прокси исправлен подсчёт попыток входа.

Как запустить

Нужны Node.js 24 и Docker Compose. В корне проекта сначала выполнить **node scripts/setup-env.mjs**, затем **docker compose up --build**. Приложение откроется по адресу http://127.0.0.1:8080/login.

Тестовые аккаунты: admin@college.test, technician@college.test, teacher@college.test. Пароль берётся из SEED_PASSWORD в созданном .env. Пароли и ключи в этот документ и Git не включены.

Где размещено

UFO Hosting, Ubuntu 24.04, 2 vCPU, 2 ГБ RAM и диск 40 ГБ. На VPS работают Caddy, Nginx, backend и PostgreSQL. Перед обновлением сохраняется копия БД. Опубликованная версия: d0158aa.

Серверные проверки CI · Проверки интерфейса CI